UNIVERSITY INSTITUTE of COMPUTING

Asia's Fastest Growing University

NAAC GRADE A+

WorkSheet -10

Student Name: Arnab Lala **UID:** 25MCI10232

Branch: M.C.A(A.I & M.L) **Section/Group:** MAM-1 A

Semester: Sem -2 **Date of Performance: 21-04-2026**

Subject Name: TECHINCAL SKILLS **Subject Code: 25CAP-652**

AIM:

To understand and apply transaction control in PostgreSQL using BEGIN, COMMIT, ROLLBACK, and SAVEPOINT to maintain data integrity during database operations.

S/W Requirement: PostgreSQL and pgAdmin

OBJECTIVES:

Apply transaction control commands for safe and reliable database updates. **Practical / Experiment Steps**

Table creation:

```
DROP TABLE IF EXISTS Payroll; CREATE TABLE Payroll (     emp_id INT PRIMARY KEY,     emp_name
VARCHAR(50),     salary DECIMAL(10,2) CHECK (salary > 0)

);

INSERT INTO Payroll VALUES

(1, 'Amit', 30000),

(2, 'Neha', 40000),

(3, 'Ravi', 50000);

SELECT * FROM Payroll;
```

Data Output Messages Notifications

	emp_id [PK] integer	emp_name character varying (50)	salary numeric (10,2)
1	1	Amit	30000.00
2	2	Neha	40000.00
3	3	Ravi	50000.00

Figure 1: Table created and initial data inserted

Experiment 1: Transaction with ROLLBACK

```
BEGIN;
UPDATE Payroll
SET salary = -1000
WHERE emp_id = 3;
SELECT * FROM Payroll;
ROLLBACK;
SELECT * FROM Payroll;
```

Data Output Messages Notifications

```
BEGIN

Query returned successfully in 53 msec.
```

Figure 2: Transaction started with BEGIN

Data Output Messages Notifications

```
ERROR:  new row for relation "payroll" violates check constraint "payroll_salary_check"
Failing row contains (3, Ravi, -1000.00).

SQL state: 23514
Detail: Failing row contains (3, Ravi, -1000.00).
```

Figure 3: Constraint violation error during update

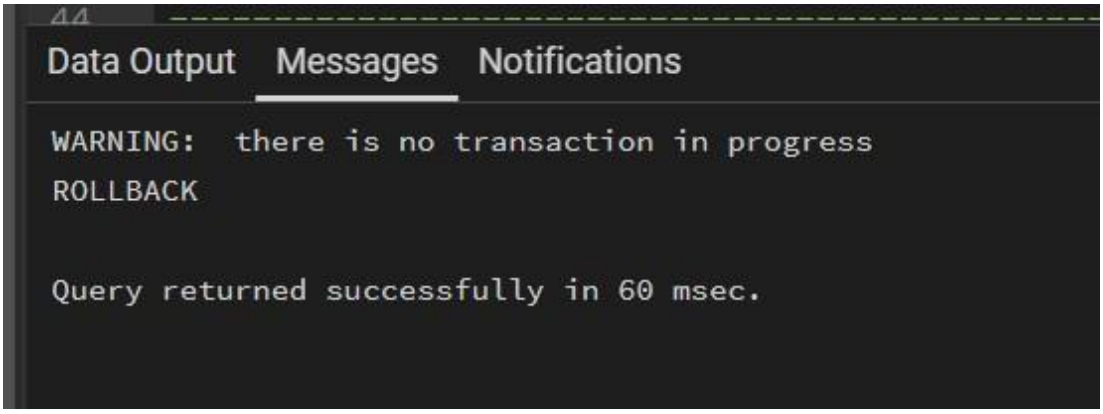

Figure 4: Table state after rollback

Experiment 2: Transaction with SAVEPOINT

```
BEGIN;
UPDATE Payroll
SET salary = salary + 5000
WHERE emp_id = 1;
SAVEPOINT sp1;
UPDATE Payroll
SET salary = salary + 7000
WHERE emp_id = 2;
UPDATE Payroll
SET salary = -1000
WHERE emp_id = 3;
ROLLBACK TO sp1;
COMMIT;
SELECT * FROM Payroll;
```

```
-- Update 1
UPDATE Payroll
SET salary = salary + 5000
WHERE emp_id = 1;

-- Savepoint created after first valid update
SAVEPOINT sp1;

-- Update 2
UPDATE Payroll
```

Data Output Messages Notifications

```
UPDATE 1

Query returned successfully in 43 msec.
```

Figure 5: First valid update before savepoint

```
WHERE emp_id = 2;
```

Data Output Messages Notifications

```
SAVEPOINT

Query returned successfully in 47 msec.
```

Figure 6: Savepoint created successfully

```sql
-- Roll back only till savepoint
ROLLBACK TO sp1;

-- Commit the valid change
COMMIT;

-- Final table
SELECT * FROM Payroll;
```

Data Output Messages Notifications

	emp_id [PK] integer	emp_name character varying (50)	salary numeric (10,2)
1	2	Neha	40000.00
2	3	Ravi	50000.00
3	1	Amit	35000.00

Figure 7: Final result after rollback to savepoint and commit

Data Output Messages Notifications

	emp_id [PK] integer	emp_name character varying (50)	salary numeric (10,2)
1	1	Amit	30000.00
2	2	Neha	40000.00
3	3	Ravi	50000.00

Figure 8: Table data after transaction handling

Outcomes:

1. **BEGIN** starts a transaction block for grouping multiple SQL commands.
2. **ROLLBACK** cancels changes made inside the transaction.
3. **SAVEPOINT** allows partial rollback without losing all valid updates.
4. **CHECK** constraint ensures salary remains greater than zero.
5. **PostgreSQL** transaction control helps maintain consistency and data integrity.